Text Analysis IV

Large Language Model-Powered Text Analysis

POLI3148 Data Science in Politics and Public Administration

Dr. Chen Haohan
The University of Hong Kong

Review: Sessions I to III

In Sessions I--III, we used what we might call **"Small" Language Models** -- methods built from relatively small text datasets:

- **Rule-based / statistical approaches:** summarize language rules and apply them algorithmically (tokenization, lemmatization, dictionary-based sentiment)
- **Supervised learning:** label a dataset, then train a classifier to learn from those labels (ML sentiment classification)
- **Unsupervised learning with strong assumptions:** no labels, but assume a data-generating process (topic modeling with LDA)

Today: a **fundamentally different approach** -- *Large* Language Models.

Part 1

What is a Large Language Model?

How LLMs Work

- Trained on massive amounts of text (books, web, code)
- Core mechanism: **predict the next word** given previous words
- "The cat sat on the ___" → "mat" (high probability)
- Through this simple task, the model learns language, facts, and reasoning

How LLMs Are Developed: From Data to Apps

Stage	What Happens	Example
1. Pre-training	Learn language patterns from trillions of tokens (web, books, code). Self-supervised: predict the next token.	GPT-3 base model
2. Supervised Fine-Tuning (SFT)	Human-written instruction-response pairs teach the model to follow instructions	InstructGPT
3. Alignment (RLHF / DPO)	Human feedback shapes responses to be helpful, harmless, honest	ChatGPT
4. Safety & Evaluation	Red-teaming, benchmarks, guardrails	Claude, GPT-4
5. Deployment	API, chat interface, app integration	OpenAI API, ChatGPT

> **Further reading:** Zhao et al. (2023+). A Survey of Large Language Models. *arXiv.* Github repo

Key Concepts

- **Token:** a piece of text. For English, ~4 characters or ~¾ of a word on average. LLMs think in tokens, not words.
- **System prompt:** instructions that define the model's behavior/role
- **User Prompt:** the input text you send to the model
- **Temperature:** controls randomness (low = focused, high = creative)
- **Context window:** how much text the model can "see" at once (8K-128K+ tokens)

> Source: OpenAI. What are tokens and how to count them? See also the interactive OpenAI Tokenizer.

LLM's Limitation: Hallucination

- LLMs predict the most **likely** next token, not the most **truthful**
- They can confidently state things that are completely wrong
  - Fabricated citations
  - Invented statistics
  - Wrong dates and names
- **For research:** LLM annotations without verification = systematic errors in your data

Mitigating Hallucination

- Always **validate** outputs
- Use **structured output** (JSON) to constrain answers
- Use **low temperature** for consistent results
- **Ground** the LLM in your data
- Supplement with traditional methods

Part 2

LLMs for Text Analysis

The Universal Text Analysis Tool

Same model + different prompts = different tasks:

- **Sentiment:** "Classify this text as positive, negative, or neutral"
- **NER:** "Extract all named entities from this text"
- **Summarization:** "Summarize this text in one sentence"
- **Topic:** "What topic is this text about?"

No feature engineering. No training data. Just natural language instructions.

Traditional vs. LLM Pipeline

Traditional Text Analysis:

Raw text → Feature Engineering (e.g., tokenize, lemmatization, stopword removal) → Construct document-term matrix (BoW approach) → Train or apply rule-based or machine learning algorithms → Get results

Large Language Model:

Raw text → Write prompt → Use prompt to interact with Large Language Model → Get results

LLM is simpler but costs money and is less transparent.

Key to Scale: Request Structured Output

- Key difference from using LLM as a chatbot or programming agent
- Free text responses are hard to process programmatically
- Basic solution: ask for **JSON output**

```
"Return your answer as JSON:
 {"sentiment": "positive", "confidence": 0.9}"
```

Structured output enables:

- **Batch processing:** run the same prompt over thousands of documents and collect results into a table
- **Automatic evaluation:** compare LLM output to a ground-truth column (precision, recall, F1)
- **Data pipelines:** LLM output becomes the input to the next step (a chart, a statistical model, a report)

What Does JSON Output Look Like?

JSON (JavaScript Object Notation) is a standard format for structured data: key-value pairs, lists, nested objects.

```
{
  "sentiment": "negative",
  "confidence": 0.85,
  "entities": ["Taiwan", "United States"],
  "reasoning": "The question contains accusatory framing."
}
```

- **Keys** (e.g., `"sentiment"`) are always in quotes
- **Values** can be strings, numbers, lists, or nested objects
- Python reads JSON into a dict: `data["sentiment"]` → `"negative"`

Further reading:

- json.org -- official JSON specification
- MDN: Working with JSON

Part 3

How to Interact with LLMs: Practical Setup

Two Ways to Use an LLM: API vs. Local

	(remote)	(on your machine)
Access	Frontier models (GPT-4o, Claude, Gemini)	Open-weight models (Llama, Gemma, Qwen)
Cost	Pay per token	Free to run, but requires hardware
Setup	API key, internet	GPU / fast CPU, model download
Privacy	Data leaves your machine	Data stays local
Speed	Fast, scalable	Depends on your hardware

Different API providers offer different gateways (OpenAI, Anthropic, Google, ...), but they share a similar interface. For teaching, we use **OpenRouter** -- a single gateway that routes to many providers through one API, including free-tier models.

OpenRouter: OUR API Gateway

- Provides access to many LLM models through **one API**
- We use **google/gemma-4-26b-a4b-it**: a small paid lite model. We avoid free-tier models because their rate limits make annotation at scale unreliable.
- Uses the OpenAI Python library (same interface, different base URL)
- Each student has an API key

API Setup

```
from openai import OpenAI

client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=OPENROUTER_API_KEY,
)

response = client.chat.completions.create(
    model="google/gemma-4-26b-a4b-it",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Your question here"}
    ],
    temperature=0.1
)
```

Cost Awareness

- We use **google/gemma-4-26b-a4b-it**, a small paid "lite" model on OpenRouter
- Pricing is in **tokens** (~4 characters of English per token), measured per *million* input/output tokens
- Lite models suitable for annotation are cheap: typically **$0.10--$2.50 per million input tokens**
- Frontier models (GPT-4o, Claude Opus, Gemini Pro) cost **5--50x more** -- usually unnecessary for classification/extraction
- Always: **start small, verify quality, then scale up**

> See the notebook (Part 7) for an end-to-end cost benchmark comparing Gemma lite, DeepSeek Flash, and Gemini Flash on the MoFA corpus.

Part 4

Prompt Engineering Basics

Writing Good Prompts

- **Provide context:** "This is a question from a diplomatic press conference"
- **Be specific:** state the task, the label set, and what each label means. For example:

> "Conduct sentiment analysis on the question. Classify its sentiment as *positive*, *negative*, or *neutral*.
> *Positive* = expresses approval, praise, or optimism.
> *Negative* = expresses criticism, accusation, or hostility.
> *Neutral* = factual or procedural, no clear positive or negative stance."

- **Define output format:** "Return JSON with keys: sentiment, confidence, reasoning"
- **Constrain the answer:** "Return ONLY the JSON output, no other text"

Zero-Shot vs. Few-Shot

Zero-shot: just describe the task, no examples

> "Classify the sentiment of this text as positive, negative, or neutral."

Few-shot: include 2-3 labeled examples

> "Here are some examples:
> Text: 'China welcomes...' → positive
> Text: 'China condemns...' → negative
> Now classify: [new text]"

Few-shot can improve accuracy for domain-specific tasks. Trade-off: more tokens per call.

Hands-On

Notebook Demo

Task 1 -- Sentiment Annotation

- Send MoFA questions to LLM with sentiment prompt
- Compare LLM labels with dataset's `q_sentiment`
- Iterate: refine instructions, add few-shot examples

Task 2 -- Named Entity Extraction

- Extract persons, organizations, locations from questions
- Compare with dataset's pre-labeled NER columns
- LLM vs. spaCy vs. dataset: three-way comparison

Task 3 -- Summarization (Single Q&A)

- Summarize individual Q&A pairs in one sentence
- Something traditional methods **cannot easily do**
- LLMs excel at generation tasks

Task 4 -- Aggregate Summarization (Topic Trees)

- Summarize a **set of Q&As** (e.g., one month of press conferences)
- Ask the LLM to return a **hierarchical topic tree** (JSON):
  - Top-level themes → sub-topics → specific issues
- **Benchmark:** compare to LDA topic modeling from Session 2
- In the notebook: visualize the tree structure

```
{"topic": "Cross-strait relations",
 "subtopics": [{"topic": "Taiwan arms sales",
                "subtopics": [...]}, ...]}
```

Exercise

Write a custom prompt to classify MoFA questions into topic categories:

- territorial_disputes, trade_and_economics, human_rights
- state_visits_and_diplomacy, military_and_security, other

Test on 5 questions. Do the labels make sense?